Threadwork: Professional Design Report

Turning `taxon-weaver` into a shared biological resolver toolkit

Draft technical report

May 11, 2026

Abstract

`taxon-weaver` is already shaped like the first member of a broader family of local-first biological resolver tools. Its strongest design choices are package-first architecture, stable JSON-like request/response contracts, deterministic-first resolution, local SQLite reference databases, lineage retrieval, reviewed decision reuse, and a thin CLI. The next strategic step is not to rewrite it from scratch, but to extract its reusable patterns into a generic **Threadwork** core so future modules such as `mesh-weaver`, `go-weaver`, `domain-weaver`, `protein-weaver`, and `gene-weaver` expose the same interface and can be integrated by downstream systems such as ORDINA without one-off glue code.

Contents

1 Executive summary **3**

2 Current state of Taxon Weaver **4**

3 Why Threadwork should exist **4**

4 Proposed ecosystem **5**

5 Core Threadwork contract **5**
- 5.1 Generic request . . . 5
- 5.2 Generic result . . . 6
- 5.3 Candidate match . . . 6
- 5.4 Ancestry entry . . . 7

6 Shared resolver interface **7**

7 Professional design solutions for current problems **7**
- 7.1 Problem: taxonomy-specific contracts . . . 7
- 7.2 Problem: reference data and reviewed decisions are mixed . . . 8
- 7.3 Problem: implicit global database path . . . 8
- 7.4 Problem: service construction initializes or mutates databases . . . 9
- 7.5 Problem: build version mixes source identity and build instance . . . 9
- 7.6 Problem: fuzzy logic is domain-specific . . . 10
- 7.7 Problem: untyped progress callbacks . . . 10
- 7.8 Problem: weak compatibility boundaries . . . 10
- 7.9 Problem: inconsistent package and import naming . . . 11
- 7.10 Problem: resolver provenance is not explicit enough . . . 11

8 Professional service composition **12**

9 Recommended package structure **12**

10 Monorepo and release strategy **13**
10.1 Recommended monorepo layout . . . 13
10.2 Individual package releases . . . 14
10.3 Umbrella package release . . . 14
10.4 Optional extras . . . 14
10.5 Release models . . . 15
10.5.1 Model A: independent versioning . . . 15
10.5.2 Model B: lockstep versioning . . . 15
10.6 Recommendation . . . 15

11 Testing standard **16**

12 Implementation roadmap **16**
12.1 Phase 1: Freeze current Taxon Weaver behavior . . . 16
12.2 Phase 2: Draft Threadwork contracts . . . 16
12.3 Phase 3: Create `threadwork-core` . . . 16
12.4 Phase 4: Adapt Taxon Weaver to implement Threadwork . . . 17
12.5 Phase 5: Split reference and decision stores . . . 17
12.6 Phase 6: Add provenance metadata to every result . . . 17
12.7 Phase 7: Build MeSH Weaver as the second module . . . 17
12.8 Phase 8: Optimize shared performance after two modules exist . . . 17

13 What should not be done **18**

14 Final recommendation **18**

1 Executive summary

The current `taxon-weaver` project should be treated as the prototype implementation of a broader resolver ecosystem. It already contains the correct instincts: the resolver is a reusable package, the CLI is thin, the database is local-first, results are JSON-serializable, fuzzy matching is supervised, and reviewed mappings can be reused conservatively.

The proposed umbrella is:

> **Threadwork: local-first resolvers for biological names, identifiers, ontologies, and curation workflows.**

The strategic shift is:

`taxon-weaver` → first concrete module in a broader `threadwork` ecosystem.

This does **not** mean making all biological logic generic. That would be a mistake. Taxonomy, MeSH, Gene Ontology, protein accessions, gene symbols, and protein domains all have different source formats, ambiguity patterns, aliases, and matching policies.

The correct professional design is:

> **Every Weaver module should feel identical to call, but remain domain-specific internally.**

Therefore, Threadwork should standardize:

- request/result/decision contracts;
- statuses, warnings, and match types;
- resolver metadata and provenance;
- batch behavior;
- reviewed decision handling;
- progress events;
- CLI conventions;
- compatibility checks;
- contract test suites.

But Threadwork should **not** force all modules to share:

- the same database schema;
- the same fuzzy matching algorithm;
- the same normalization rules;
- the same ontology traversal rules;
- the same ambiguity policy.

The recommended implementation route is to create a small `threadwork-core` package containing only the generic contracts, enums, metadata, protocols, progress events, and test utilities. Then `taxon-weaver` should be refactored to implement those contracts while keeping taxonomy-specific logic inside its own package. After that, `mesh-weaver` should be built as the second real module, because ORDINA needs disease normalization and because a second module will test whether the abstraction is truly useful.

2 Current state of Taxon Weaver

`taxon-weaver` is already more than a single-purpose script. It is a local-first resolver package for NCBI taxonomy names. The current design includes:

- a reusable core library;
- a thin command-line interface;
- local SQLite reference database construction from NCBI taxdump files;
- exact scientific-name lookup;
- synonym lookup;
- normalized-name lookup;
- transform-assisted fallback;
- supervised fuzzy suggestions;
- lineage retrieval;
- reviewed mapping persistence;
- conservative cache reuse;
- stable dataclass contracts;
- JSON-ready outputs;
- build metadata for reproducibility.

The important design lesson is that a resolver should behave like a local API even when imported directly as a Python package. That means downstream applications can call the resolver through structured contracts instead of depending on implementation details.

```
Downstream application
  -> ResolveRequest
  -> Weaver module
  -> ResolveResult
```

This is the right pattern for ORDINA, CLI workflows, Celery workers, Django services, local scripts, and future HTTP APIs.

3 Why Threadwork should exist

Without a shared contract, every future resolver will repeat the same design decisions:

- How does a caller ask for a value to be resolved?
- How are exact, synonym, normalized, fuzzy, ambiguous, and unresolved matches represented?
- How are candidates represented?
- How are warnings stored?
- How does a curator confirm or reject a candidate?
- How are prior decisions reused?
- How is source-build metadata recorded?
- How does a downstream application know whether a result can be auto-accepted?

If `taxon-weaver`, `mesh-weaver`, `go-weaver`, and future modules solve these questions differently, ORDINA will accumulate one-off adapters. That would make the broader system harder to maintain and harder to test.

Threadwork should exist to define the common resolver language. Each Weaver module should still own its biological logic, but all modules should present the same outer interface.

4 Proposed ecosystem

The proposed package family is:

```
threadwork-core    Generic contracts, protocols, metadata, test helpers
threadwork         Optional umbrella package / compatibility bundle

taxon-weaver       NCBI taxonomy resolver
mesh-weaver        MeSH disease/medical descriptor resolver
go-weaver          Gene Ontology resolver
domain-weaver      Protein domain resolver, e.g. Pfam/InterPro-like sources
protein-weaver     Protein/accession resolver, e.g. UniProt-like sources
gene-weaver        Gene symbol/identifier resolver
```

The relationship should be:

```
threadwork-core
  provides contracts and interfaces

individual Weaver modules
  implement those interfaces using source-specific logic

threadwork umbrella package
  optionally installs compatible sets of Weaver modules
```

For ORDINA, this gives a clean normalization layer:

```
ORDINA Import Worker
  -> Threadwork ResolveRequest
  -> taxon-weaver / mesh-weaver / go-weaver
  -> Threadwork ResolveResult
  -> candidate evidence / review queue / canonical entity link
```

5 Core Threadwork contract

The current Taxon Weaver contracts are a strong starting point, but they use taxonomy-specific field names. The shared contract should be generic.

5.1 Generic request

```
ResolveRequest
  original_value: str
  namespace: str
  provided_type: str | None
  allow_fuzzy: bool
  source: dict
  context: dict
  options: dict
```

For Taxon Weaver:

```
original_value  -> organism name as written
provided_type   -> provided taxonomic rank, e.g. species/genus/family
namespace       -> ncbi_taxonomy
```

For MeSH Weaver:

```
original_value  -> disease phrase
provided_type   -> descriptor/category if known
namespace       -> mesh
```

For GO Weaver:

```
original_value  -> GO label, synonym, or GO identifier
provided_type   -> biological_process/molecular_function/cellular_component
namespace       -> gene_ontology
```

5.2 Generic result

```
ResolveResult
  original_value: str
  normalized_value: str
  namespace: str

  status: ResolutionStatus
  review_required: bool
  auto_accept: bool
  match_type: MatchType
  warnings: list[WarningCode]

  matched_id: str | None
  matched_label: str | None
  matched_type: str | None
  score: float | None

  candidates: list[CandidateMatch]
  ancestry: list[AncestryEntry]
  aliases: list[AliasEntry]

  cache_applied: bool
  resolver: ResolverMetadata
  metadata: dict
```

For taxonomy:

```
matched_id       -> NCBI TaxID
matched_label    -> scientific name
matched_type     -> rank
ancestry         -> lineage
aliases          -> synonyms / alternative names
```

5.3 Candidate match

```
CandidateMatch
  entity_id: str
  label: str
  entity_type: str | None
  namespace: str
  match_type: MatchType
  score: float | None
  ancestry: list[AncestryEntry]
  aliases: list[AliasEntry]
  warnings: list[WarningCode]
  metadata: dict
```

5.4 Ancestry entry

Taxonomy has lineage. GO has ontology parents. MeSH has tree positions. Protein/domain systems may have cross-reference chains. A shared term like `ancestry` is more general than `lineage`.

```
AncestryEntry
  entity_id: str
  label: str
  entity_type: str | None
  relationship: str | None
  depth: int | None
  metadata: dict
```

For taxonomy, `relationship` can be `parent`. For GO, it may be `is`$_a or$`part`$_o f. For MeSH, it may be$`tree`$_p aren$

5.5 Decision record

```
DecisionRecord
  action: DecisionAction
  original_value: str
  normalized_value: str
  provided_type: str | None
  namespace: str
  source_version: str
  reviewer: str | None

  resolved_id: str | None
  resolved_label: str | None
  resolved_type: str | None

  match_type: MatchType
  status: ResolutionStatus
  score: float | None
  warnings: list[WarningCode]
  notes: str | None
  created_at: str
```

This lets all Weaver modules share the same review and cache logic.

6 Shared resolver interface

Every Weaver module should expose the same service methods:

```
class ResolverService:
    def resolve_one(request: ResolveRequest) -> ResolveResult: ...
    def resolve_batch(request: BatchResolveRequest) -> BatchResolveResult: ...
    def get_ancestry(entity_id: str) -> list[AncestryEntry]: ...
    def get_build_info() -> BuildInfo: ...
    def record_decision(decision: DecisionRecord) -> None: ...
    def close() -> None: ...
```

The implementation can be domain-specific, but the interface should not be.

7 Professional design solutions for current problems

7.1 Problem: taxonomy-specific contracts

The current names are readable for taxonomy but should not become the generic standard.

Solution: define generic Threadwork contracts and preserve taxonomy aliases.

Taxon Weaver concept	Threadwork concept
original_name	original_value
normalized_name	normalized_value
provided_level	provided_type
matched_taxid	matched_id
matched_name	matched_label
matched_rank	matched_type
lineage	ancestry
taxonomy_build_version	source_version

Taxon-specific aliases can remain available for ergonomics, but new downstream code should depend on the generic contract.

7.2 Problem: reference data and reviewed decisions are mixed

Reference data and reviewed decisions have different lifecycles. Reference data is rebuilt from source dumps. Reviewed decisions are user/application state.

Solution: split storage into a read-mostly ReferenceStore and a writable DecisionStore.

```
ReferenceStore
  exact_lookup(...)
  alias_lookup(...)
  normalized_lookup(...)
  get_ancestry(...)
  get_aliases(...)
  get_build_info(...)
  check_compatibility(...)

DecisionStore
  lookup(...)
  record(...)
```

For the MVP, both stores may be SQLite. The key improvement is that they are separate responsibilities.

7.3 Problem: implicit global database path

A global default database path is convenient for small scripts but risky in tests, workers, web servers, and long-lived applications.

Solution: require an explicit resolver context in library code.

```
ResolverContext
  reference_path: Path
  decision_path: Path | None
  namespace: str
  read_only_reference: bool
  options: dict
```

The CLI can still infer paths from command-line arguments or environment variables, but the core library should prefer explicit dependencies.

7.4 Problem: service construction initializes or mutates databases

A resolver service should not silently create or mutate a reference database when it is opened.

Solution: separate build, validate, runtime, and decision phases.

```
Build phase:
  create reference database from source files

Validate phase:
  check schema version, source version, indexes, and metadata

Runtime phase:
  open reference database read-only and resolve values

Decision phase:
  write user decisions to a separate decision store
```

A professional API should look like:

```
TaxonReferenceBuilder.build(...)
TaxonReferenceStore.open_readonly(...)
TaxonReferenceStore.validate()
TaxonResolverService(reference_store=..., decision_store=...)
```

7.5 Problem: build version mixes source identity and build instance

A timestamped build version can make decision reuse stricter than necessary if the same source dump is rebuilt on a different date.

Solution: split version concepts.

```
source_version
  identity of the external source data
  example: ncbi-taxdump-sha256:<hash>

schema_version
  identity of the local reference schema
  example: taxon-reference-schema-v2

resolver_version
  software package version
  example: taxon-weaver 0.2.0

policy_version
  identity of matching/review policy
  example: taxon-resolution-policy-v1

build_instance_id
  timestamp/environment of this specific DB build
```

Decision reuse should depend mostly on:

```
namespace + normalized_value + provided_type + source_version + policy_version
```

not only on a timestamped build ID.

7.6 Problem: fuzzy logic is domain-specific

Taxon Weaver fuzzy matching is correctly optimized for scientific names, ranks, and taxonomy-like strings. That logic should not become the default for all Threadwork modules.

Solution: define a fuzzy strategy interface.

```
class FuzzyStrategy:
    def suggest(
        request: ResolveRequest,
        store: ReferenceStore,
    ) -> list[CandidateMatch]: ...
```

Then each module implements its own strategy:

- TaxonFuzzyStrategy: scientific names, synonyms, ranks, genus/species token behavior;
- MeshFuzzyStrategy: disease descriptors, entry terms, abbreviations;
- GOFuzzyStrategy: GO labels, synonyms, ontology branch constraints;
- ProteinFuzzyStrategy: accessions, gene symbols, protein names.

Threadwork should standardize the result shape, not the biological scoring logic.

7.7 Problem: untyped progress callbacks

Build progress should be useful for CLI output, Celery task progress, web dashboards, logs, and future APIs.

Solution: use typed progress events.

```
ProgressEvent
  stage: str
  message: str
  current: int | None
  total: int | None
  final: bool
  severity: str
  metadata: dict
```

This fits future ORDINA workers because the same event can update a Celery job, a database status row, or a web UI.

7.8 Problem: weak compatibility boundaries

Reference stores and decision stores need different compatibility policies.

Solution: add explicit compatibility reports.

```
CompatibilityReport
  ok: bool
  schema_version: str
  expected_schema_version: str
  source_version: str
  warnings: list[str]
  errors: list[str]
```

Recommended policy:

- if a reference schema is incompatible, rebuild the reference database from source;

- if a decision schema is incompatible, migrate carefully;
- user decisions are not reproducible, so they deserve more care than reference data;
- the resolver should fail clearly rather than guessing.

7.9 Problem: inconsistent package and import naming

A professional ecosystem should have predictable package names.

Solution: define canonical names and compatibility imports.

Canonical names:

```
threadwork_core
taxon_weaver
taxon_weaver_cli
mesh_weaver
go_weaver
domain_weaver
protein_weaver
gene_weaver
```

Canonical import:

```
from threadwork_core import ResolveRequest
from taxon_weaver import TaxonResolverService
```

Compatibility import:

```
from taxonomy_resolver import ResolveRequest, TaxonomyResolverService
```

Compatibility imports can remain for a while, but new code should use the new names.

7.10 Problem: resolver provenance is not explicit enough

ORDINA and similar downstream systems need to know which resolver produced a match, using which source data and which policy.

Solution: add resolver metadata to every result.

```
ResolverMetadata
  resolver_id: str
  resolver_version: str
  namespace: str
  source_name: str
  source_version: str
  schema_version: str
  policy_version: str
```

Example:

```
{
  "resolver_id": "taxon-weaver",
  "resolver_version": "0.2.0",
  "namespace": "ncbi_taxonomy",
  "source_name": "NCBI taxdump",
  "source_version": "sha256:...",
  "schema_version": "taxon-reference-v2",
  "policy_version": "taxon-resolution-policy-v1"
}
```

This allows ORDINA to keep scientific provenance instead of only storing the final matched identifier.

8 Professional service composition

The most professional runtime pattern is dependency composition:

```
resolver = TaxonResolverService(
    reference_store=TaxonReferenceStore.open_readonly(reference_path),
    decision_store=SQLiteDecisionStore(decision_path),
    normalizer=TaxonNormalizer(),
    fuzzy_strategy=TaxonFuzzyStrategy(),
    policy=TaxonResolutionPolicy(),
)
```

A convenience constructor can still exist:

```
resolver = TaxonResolverService.from_paths(
    reference_path="data/ncbi_taxonomy.sqlite",
    decision_path="data/taxon_decisions.sqlite",
)
```

The important rule is that the simple constructor should be a wrapper over explicit components, not the only design.

9 Recommended package structure

A cleaner package layout would be:

```
threadwork-core/
  src/threadwork_core/
    contracts.py
    enums.py
    protocols.py
    metadata.py
    serialization.py
    progress.py
    errors.py
    testing.py

taxon-weaver/
  src/taxon_weaver/
    service.py
    reference_store.py
    decision_store.py
    builder.py
    normalize.py
    exact.py
    fuzzy.py
    transforms.py
    policy.py
    lineage.py
    manifest.py

  src/taxon_weaver_cli/
    cli.py
    build_reference.py
    resolve_one.py
    resolve_batch.py
    inspect_entity.py
    apply_decisions.py
```

```
  tests/
    test_contract_shape.py
    test_exact_resolution.py
    test_fuzzy_resolution.py
    test_decision_reuse.py
    test_reference_build.py
```

The goal is to prevent generic contracts, CLI logic, source-building logic, taxonomy-spec policy, and decision persistence from drifting together.

10 Monorepo and release strategy

Threadwork can contain many packages in one repository. This is a normal and professional approach when several packages share a contract and are expected to evolve together.

10.1 Recommended monorepo layout

```
threadwork/
  packages/
    threadwork-core/
      pyproject.toml
      src/threadwork_core/

    taxon-weaver/
      pyproject.toml
      src/taxon_weaver/
      src/taxon_weaver_cli/

    mesh-weaver/
      pyproject.toml
      src/mesh_weaver/
      src/mesh_weaver_cli/

    go-weaver/
      pyproject.toml
      src/go_weaver/
      src/go_weaver_cli/

    threadwork/
      pyproject.toml
      src/threadwork/

  docs/
    architecture.md
    contracts.md
    release-policy.md

  examples/
    ordina_integration/
    cli_examples/

  tests/
    integration/
    contract/
```

Each package has its own pyproject.toml. That means each package can be built, versioned, tested, and released independently.

10.2 Individual package releases

Each Weaver can have its own independent release:

```
threadwork-core 0.3.0
taxon-weaver    0.4.1
mesh-weaver     0.1.0
go-weaver       0.1.0
```

This is useful because the modules will not mature at the same speed. Taxon Weaver may become stable before MeSH Weaver, while GO Weaver may need several schema revisions.

A downstream project such as ORDINA could depend only on what it needs:

```
ordina dependencies:
  threadwork-core >=0.3,<0.4
  taxon-weaver >=0.4,<0.5
  mesh-weaver >=0.1,<0.2
```

10.3 Umbrella package release

Threadwork can also provide a convenience package:

```
threadwork
```

This package should not duplicate code. It should depend on compatible versions of the individual packages.

Example:

```
threadwork 0.2.0
  depends on:
    threadwork-core >=0.3,<0.4
    taxon-weaver >=0.4,<0.5
    mesh-weaver >=0.1,<0.2
    go-weaver >=0.1,<0.2
```

Users who want everything can install:

```
pip install threadwork
```

Users who only need taxonomy can install:

```
pip install taxon-weaver
```

10.4 Optional extras

Another good pattern is optional extras:

```
pip install threadwork[taxon]
pip install threadwork[mesh]
pip install threadwork[go]
pip install threadwork[all]
```

A possible umbrella package configuration:

```
[project.optional-dependencies]
taxon = ["taxon-weaver>=0.4,<0.5"]
mesh = ["mesh-weaver>=0.1,<0.2"]
go = ["go-weaver>=0.1,<0.2"]
all = [
  "taxon-weaver>=0.4,<0.5",
  "mesh-weaver>=0.1,<0.2",
  "go-weaver>=0.1,<0.2",
]
```

10.5 Release models

There are two realistic release models.

10.5.1 Model A: independent versioning

Each package has its own version:

```
threadwork-core 0.3.0
taxon-weaver    0.4.1
mesh-weaver     0.1.0
threadwork      0.2.0
```

Pros:

- flexible;
- avoids releasing unchanged packages;
- better when modules mature at different speeds.

Cons:

- dependency compatibility must be documented carefully;
- release automation is slightly more complex.

10.5.2 Model B: lockstep versioning

All packages share the same version:

```
threadwork-core 0.4.0
taxon-weaver    0.4.0
mesh-weaver     0.4.0
threadwork      0.4.0
```

Pros:

- easy to understand;
- simple compatibility story.

Cons:

- forces releases of unchanged packages;
- awkward when one module is experimental and another is stable.

10.6 Recommendation

Use **independent package versioning**, with an umbrella threadwork release that pins compatible package ranges.

Recommended policy:

- threadwork-core follows semantic versioning strictly;
- each Weaver module declares which threadwork-core version range it supports;
- the umbrella threadwork package is a convenience compatibility bundle;
- breaking contract changes require a new major or minor line in threadwork-core;
- ORDINA should depend on explicit package ranges, not floating latest versions.

11 Testing standard

Threadwork should have contract tests, not only behavior tests. If ORDINA depends on a field, that field should be covered by a contract test.

Minimum test set:

- every result serializes cleanly to JSON;
- every status is documented;
- every warning is documented;
- exact match result shape is stable;
- synonym match result shape is stable;
- fuzzy suggestion result shape is stable;
- unresolved result shape is stable;
- cached decision result shape is stable;
- compatibility imports still work;
- reference-store compatibility checks fail clearly;
- decision reuse does not depend only on timestamped build IDs;
- each Weaver module passes the shared Threadwork contract test suite.

12 Implementation roadmap

12.1 Phase 1: Freeze current Taxon Weaver behavior

Before extracting anything, freeze the current behavior with tests.

Deliverables:

- contract-shape tests for current results;
- examples for exact, synonym, normalized, fuzzy, unresolved, cached, and decision cases;
- documented compatibility expectations.

12.2 Phase 2: Draft Threadwork contracts

Create a design document defining:

- generic dataclasses;
- statuses;
- warnings;
- resolver interface;
- reference-store interface;
- decision-store interface;
- build metadata;
- compatibility rules.

12.3 Phase 3: Create threadwork-core

Start small. The first version should contain only:

- contracts;
- enums;
- protocols;
- metadata models;

- progress events;
- serialization helpers;
- test helpers.

It should not contain taxonomy logic.

12.4 Phase 4: Adapt Taxon Weaver to implement Threadwork

Taxon Weaver should import threadwork-core and map taxonomy fields to generic fields.

Deliverables:

- TaxonResolverService implementing the generic resolver interface;
- compatibility aliases for old Taxon Weaver field names;
- unchanged CLI behavior where possible;
- contract compatibility tests.

12.5 Phase 5: Split reference and decision stores

Refactor storage into:

- TaxonomyReferenceStore;
- SQLiteDecisionStore;
- explicit read-only reference opening;
- explicit decision-store initialization.

12.6 Phase 6: Add provenance metadata to every result

Every result should include:

- resolver ID;
- resolver version;
- namespace;
- source name;
- source version;
- schema version;
- policy version.

12.7 Phase 7: Build MeSH Weaver as the second module

MeSH Weaver is the best second module because ORDINA needs disease normalization.
It will also reveal whether the Threadwork abstraction is real or accidentally taxonomy-spec

Deliverables:

- MeSH reference builder;
- descriptor/entry-term resolver;
- same Threadwork contract;
- simple CLI;
- ORDINA integration proof of concept.

12.8 Phase 8: Optimize shared performance after two modules exist

Do not over-engineer Threadwork before a second module exists. Once Taxon Weaver and MeSH Weaver both use the contract, optimize shared pieces.

Possible targets:

- batch resolution streaming;
- shared SQLite connection handling;
- optional multiprocessing for large batches;
- shared progress events;
- shared test fixtures;
- optional full-text-search experiments where justified.

13 What should not be done

Avoid these traps:

- Do not rewrite Taxon Weaver from scratch before freezing the current contract.
- Do not make threadwork-core depend on taxonomy concepts.
- Do not force all modules to use identical database schemas.
- Do not build a microservice before the package interface is stable.
- Do not let ORDINA query domain-specific reference tables directly.
- Do not optimize a generic fuzzy algorithm before each domain has its own policy.
- Do not use the umbrella threadwork package as the only installation path.

14 Final recommendation

Taxon Weaver should remain the first concrete Weaver module, but it should no longer be the only design target. The best next move is to turn its strongest design ideas into a generic Threadwork contract while keeping taxonomy-specific behavior inside Taxon Weaver.

The practical path is:

1. keep Taxon Weaver working;
2. create threadwork-core with contracts and protocols only;
3. adapt Taxon Weaver to implement those contracts;
4. split reference storage from decision storage;
5. add resolver/source/schema/policy provenance;
6. use a monorepo with independently versioned packages;
7. add an umbrella threadwork package for convenience;
8. build MeSH Weaver as the second real module;
9. use ORDINA as the first serious downstream integration.

The guiding principle should remain:

same outer interface, different inner biology.